CHALLENGE ORACLE

Rock New Ventures: Oportunidades no ecossistema de inovação

ORACLE

Rock New Ventures

STARTING UP THE FUTURE

Calendário

23/03 - Mentoria com Oracle na FIAP

10/04 - Sprint 3

18/05 - Banca com professores na FIAP

21/05 - Sprint 4

22/06 - Banca Final de Aprovação para o NEXT na Oracle

24/10 - NEXT

Entregáveis

Sprint 3

COMPLIANCE & QUALITY ASSURANCE

2TDS Entrega Sprint 3

Desenvolva o plano detalhado do seu projeto Challenge, usando o AZURE BOARDS, contendo:

- Estrutura de trabalho em forma de Backlog de Produto, com detalhes de Épicos, Features e Product Backlog Itens, no padrão Scrum (peso 20%);
- Documentação de descrição e critérios de aceite, explicando as histórias de usuário formadas pelos Épicos, Features e Product Backlog Itens (peso 20%);
- Apontamento da Prioridade, Esforço, Dependência entre itens do backlog, ordenando o backlog na sequência de implementação e entrega (peso 20%);
- Release plan, contendo a indicação de qual a Sprint que desenvolverá e entregará cada item do backlog, criando um roadmap de entregas, balanceando os pontos de esforço entre Sprints (peso 20%);
- Detalhamento da Sprint atual do projeto (em curso), com Tarefas detalhadas com suas descrições, pontuações de esforço e dependências técnicas (20%).

Entregue o link de acesso ao seu plano de projetos na nuvem.
Garanta que o professor esteja cadastrado como membro da organização e administrador do

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Tarefas e pontuação: Sql Oracle - Pontuação

Desenvolver 02 procedimentos (até 25 pontos)
Duas funções (até 25 pontos)
Um gatilho (até 20 pontos)
Documentação (até 10 pontos) - MLR + MF – Oracle data modeler
Video (até 20 pontos)

A avaliação contempla o grau de complexidade de cada item acima citado, quanto mais simples menor a pontuação.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Tarefa procedimento:

- ⑩ Cada procedimento deve tratar três exceções diferentes .

- ⑩ O primeiro procedimento deverá conter uma junção de duas ou mais tabelas relacionadas e exibir o resultado do processamento obtido no formato JSON. Os dados devem ser transformados do formato relacional para o formato JSON através de uma função desenvolvida pelo grupo (vide requisto 2). Cada tabela do projeto deve ter, no mínimo cinco registros.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Tarefa procedimento:

- o segundo procedimento deve ler os dados de uma tabela e, na mesma linha, mostrar o valor de uma coluna da linha atual, o valor dessa mesma coluna na linha anterior e o valor dessa mesma coluna na próxima linha. Caso a linha anterior ou a próxima linha não existir, apresentar a palavra "Vazio". O relatório deve ter, pelo menos, cinco linhas de dados. A tabela e a coluna a ser exibida fica a cargo do grupo.

⑩Vide exemplo

Cod_emp	Cod_dep	Anterior	Atual	Próximo
1	10	Vazio	3000	3500
2	10	3000	3500	4000
3	10	3500	4000	Vazio

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Tarefa função:

- Cada função deverá tratar três exceções diferentes.
- Uma função deve ler os dados recebidos e transformá-lo para o formato JSON. Não use as funções built-in internas de transformação para JSON do banco de dados Oracle. Essa função será usada no primeiro procedimento solicitado)

- Uma função deve substituir um dos processamentos existentes em seu projeto , como por exemplo verificação da complexidade da senha ou cálculos Matemáticos.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Tarefa gatilho:

- Crie uma tabela de auditoria. A cada INSERT, UPDATE ou DELETE em uma tabela escolhida pelo grupo essa tabela deverá ser atualizada com os dados anteriores (OLD) e os novos dados (NEW) da tabela, com o nome do usuário que fez a operação, qual o nome da operação (INSERT, UPDATE
ou DELETE) e a data em que a operação foi realizada.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Entrega:

- Arquivo PDF contendo folha de rosto com os nomes completos dos integrantes do time em ordem
alfébética e seus RMS.
- Arquivo com a codificação SQL solicitada (Tabelas, Dados, Procedimentos, Funções e Triggers)
- Vídeo (com áudio) demonstrando a execução dos procedimentos, funções e gatilhos.

Obs: A boa organização do conteúdo do arquivo é de suma importância para apontamento da nota, caso haja falha nesta organização a correção pode vir a ser comprometida acarretando diminuição da nota.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Penalidades:

- A falta da documentação compromete totalmente a correção das instruções em SQL, acarretando uma nota mínima de entrega de 30% do total da atividade.
- O não cumprimento dos requisitos solicitados na codificação implica na perda de 50% do total da nota do item.
- A criação de uma ou mais instruções que não atendam o solicitado será desconsiderada, sem valor na pontuação.
- A instrução que atenda parcialmente o solicitado terá como valor 50% do total deste item.
- Instruções que apresentarem problemas de execução serão desconsideradas e não haverá pontuação para a mesma.
- Video de baixa resolução, sem áudio e faltando explicação será desconsiderado.

DEVOPS TOOLS E CLOUD COMPUTING 1/2

Utilizando uma das tecnologias aprendidas: Serviços de Aplicativos e Banco de Dados (deve-se utilizar ambos):

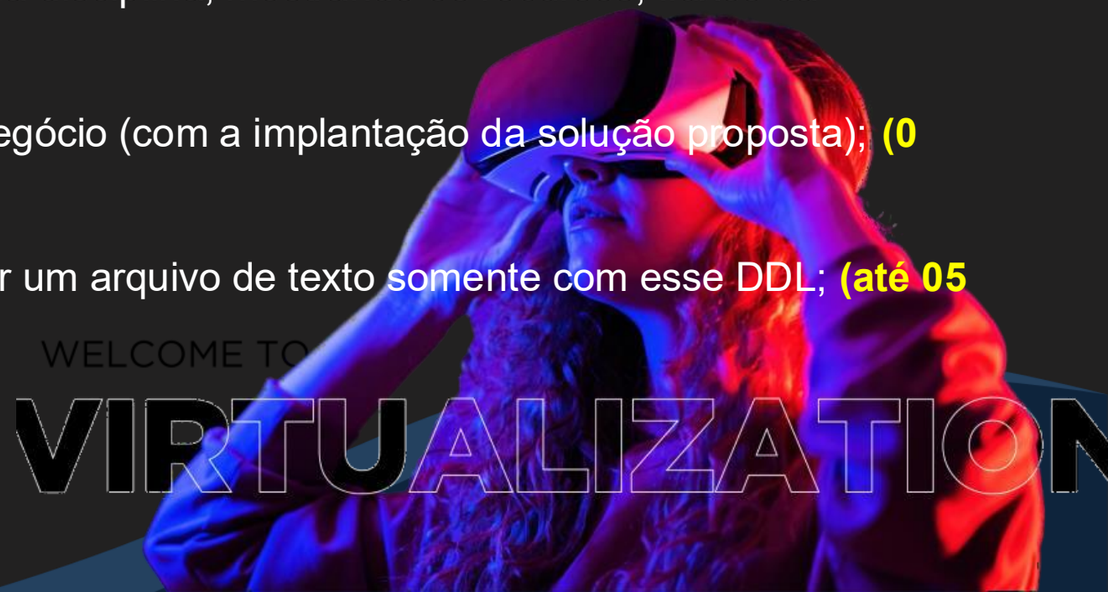
- Implantar a solução ou parte dela (um módulo do sistema) da Odontoprev em Nuvem;
- Serão aceitas soluções com Front ou API;
- Obrigatório um Banco de Dados em Nuvem que não seja o H2. Serão aceitos: Oracle, MySQL, SQL da Azure ou PostgreSQL;
- Realize um CRUD (Inclusão, Alteração, Exclusão e Consulta) sobre **pelo menos duas tabelas com um relacionamento entre elas;**

1.Uma breve descrição de sua solução; **(0 pontos – Na falta a nota será subtraída em 05 pontos);**

2.O desenho da arquitetura da solução proposta (desenho baseado em nossa disciplina, mostrando os recursos, fluxos da informação etc); **(até 20 pontos);**

3.Uma breve descrição dos benefícios a serem alcançados em relação ao negócio (com a implantação da solução proposta); **(0 pontos – Na falta a nota será subtraída em 05 pontos);**

4.DDL das tabelas (tabelas, colunas, chave primária, comentários etc) - Criar um arquivo de texto somente com esse DDL; **(até 05 pontos);**



DEVOPS TOOLS E CLOUD COMPUTING 2/2

5.Código-Fonte da aplicação no Github (e com tudo que é necessário para a execução do App/API) com um Readme.md para realizar o Deploy e testes (O Prof. irá reproduzir e testar o deploy com base nesse Readme.md). Incluir os scripts JSON do CRUD, se a solução for uma API; **(até 05 pontos);**

6.Uma gravação de um vídeo mostrando o funcionamento da solução, desde o Clone do Repositório (ou do **build da Imagem), mostrando o funcionamento até persistência de dados em Nuvem MS Azure. Inclusive, o** vídeo deve mostrar os dados persistidos no banco de dados (CRUD); **(até 70 pontos);**

Obs: O vídeo precisa ter qualidade de no mínimo 720p, e conter áudio ou legenda explicando o desenvolvimento da solução em nuvem.

7.Arquivo PDF, contendo o nome e RM dos integrantes do time, link do Github e Link do Vídeo. **(0 pontos);**

Obs: O professor precisa ter acesso ao vídeo.

A PONTUAÇÃO DE CADA QUESITO VAI VARIAR DE ACORDO COM A QUALIDADE DA TAREFA ENTREGUE

ADVANCED BUSINESS DEVELOPMENT WITH .NET

Você deverá evoluir a aplicação ASP.NET Core desenvolvida nas sprints anteriores, adicionando camadas de monitoramento, observabilidade e testes automatizados. Esta sprint tem foco nos seguintes tópicos:

- Monitoramento e Observabilidade;
- Testes Automatizados com padrão AAA;
- Atualização da documentação do projeto.

Requisitos

1. Implementação de Monitoramento e Observabilidade (40 pts)

- Health Checks - Implementar endpoints de Health Check para verificar a saúde da API, conectividade com o banco de dados (Oracle/MongoDB) e disponibilidade de serviços externos. Utilizar Microsoft.Extensions.Diagnostics.HealthChecks. (15 pts)

- Logging Estruturado - Configurar logging estruturado com Serilog ou NLog, incluindo níveis de log (Information, Warning, Error), correlação de requisições e saída para console/arquivo. (10 pts)

- Tracing e Métricas: Implementar distributed tracing com OpenTelemetry ou Application Insights para rastrear requisições entre camadas da aplicação. Expor métricas de desempenho (tempo de resposta, taxa de erros). (15 pts)

ADVANCED BUSINESS DEVELOPMENT WITH .NET

2. Implementação de Testes Automatizados – Padrão AAA (50 pts)

- Testes Unitários com xUnit: Implementar testes unitários seguindo o padrão AAA (Arrange, Act, Assert) para as camadas de Domínio e Aplicação. Utilizar xUnit como framework de testes, com Moq ou NSubstitute para mocking de dependências. (20 pts)

- Testes de Integração: Implementar testes de integração para os endpoints da API utilizando WebApplicationFactory. Validar o fluxo completo de requisições HTTP, incluindo autenticação, respostas de sucesso e tratamento de erros. (15 pts)

- Cobertura e Organização dos Testes: Organizar os projetos de teste separados por camada (Unit, Integration). Garantir nomenclatura consistente dos testes (MetodoTestado_Cenario_ResultadoEsperado). Utilizar Fixtures e Collection Fixtures para compartilhar contexto entre testes. (15 pts)

3. Atualização do README (10 pts)

- Documentar os endpoints de Health Check e como monitorar a aplicação.
- Incluir instruções de como executar os testes (dotnet test).
- Atualizar a descrição geral do projeto com as novas funcionalidades.

DEVOPS TOOLS E CLOUD COMPUTING 2/2

5.Código-Fonte da aplicação no Github (e com tudo que é necessário para a execução do App/API) com um Readme.md para realizar o Deploy e testes (O Prof. irá reproduzir e testar o deploy com base nesse Readme.md). Incluir os scripts JSON do CRUD, se a solução for uma API; **(até 05 pontos);**

6.Uma gravação de um vídeo mostrando o funcionamento da solução, desde o Clone do Repositório (ou do **build da Imagem), mostrando o funcionamento até persistência de dados em Nuvem MS Azure. Inclusive, o** vídeo deve mostrar os dados persistidos no banco de dados (CRUD); **(até 70 pontos);**

Obs: O vídeo precisa ter qualidade de no mínimo 720p, e conter áudio ou legenda explicando o desenvolvimento da solução em nuvem.

7.Arquivo PDF, contendo o nome e RM dos integrantes do time, link do Github e Link do Vídeo. **(0 pontos);**

Obs: O professor precisa ter acesso ao vídeo.

A PONTUAÇÃO DE CADA QUESITO VAI VARIAR DE ACORDO COM A QUALIDADE DA TAREFA ENTREGUE

ADVANCED BUSINESS DEVELOPMENT WITH .NET

Você deverá evoluir a aplicação ASP.NET Core desenvolvida nas sprints anteriores, adicionando camadas de monitoramento, observabilidade e testes automatizados. Esta sprint tem foco nos seguintes tópicos:

- Monitoramento e Observabilidade;
- Testes Automatizados com padrão AAA;
- Atualização da documentação do projeto.

Requisitos

1. Implementação de Monitoramento e Observabilidade (40 pts)

- Health Checks - Implementar endpoints de Health Check para verificar a saúde da API, conectividade com o banco de dados (Oracle/MongoDB) e disponibilidade de serviços externos. Utilizar Microsoft.Extensions.Diagnostics.HealthChecks. (15 pts)
- Logging Estruturado - Configurar logging estruturado com Serilog ou NLog, incluindo níveis de log (Information, Warning, Error), correlação de requisições e saída para console/arquivo. (10 pts)
- Tracing e Métricas: Implementar distributed tracing com OpenTelemetry ou Application Insights para rastrear requisições entre camadas da aplicação. Expor métricas de

ADVANCED BUSINESS DEVELOPMENT WITH .NET

2. Implementação de Testes Automatizados – Padrão AAA (50 pts)

- Testes Unitários com xUnit: Implementar testes unitários seguindo o padrão AAA (Arrange, Act, Assert) para as camadas de Domínio e Aplicação. Utilizar xUnit como framework de testes, com Moq ou NSubstitute para mocking de dependências. (20 pts)
- Testes de Integração: Implementar testes de integração para os endpoints da API utilizando WebApplicationFactory. Validar o fluxo completo de requisições HTTP, incluindo autenticação, respostas de sucesso e tratamento de erros. (15 pts)
- Cobertura e Organização dos Testes: Organizar os projetos de teste separados por camada (Unit, Integration). Garantir nomenclatura consistente dos testes (MetodoTestado_Cenario_ResultadoEsperado). Utilizar Fixtures e Collection Fixtures para compartilhar contexto entre testes. (15 pts)

3. Atualização do README (10 pts)

- Documentar os endpoints de Health Check e como monitorar a aplicação.
- Incluir instruções de como executar os testes (dotnet test).
- Atualizar a descrição geral do projeto com as novas funcionalidades.

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Objetivos Principais

- Definir e documentar o componente de Inteligência Artificial que será integrado à aplicação Oracle APEX do Challenge, incluindo caso de uso, dados, modelo de IA e estratégia de integração com o APEX/Oracle Database.
- Gravar um vídeo pitch de aproximadamente 5 minutos com o modelo de IA e estratégia de integração com o APEX/Oracle Database

Objetivos Específicos

- Definir o problema de IA do projeto dentro da aplicação APEX
- Demonstrar o funcionamento de **componentes-chave** do projeto.
- Descrever o fluxo de dados entre APEX, Oracle Database e a IA
- Definir os dados que serão usados na IA
- Escolher e justificar o tipo de modelo de IA

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Tarefas

- Definir qual problema de IA o projeto irá resolver e como essa IA será usada dentro da aplicação APEX.
- Escolher o modelo de IA que será utilizado (CNN, Transfer Learning, NLP, LLM etc.) e justificar a escolha.
- Identificar e descrever quais dados serão usados para treinar ou alimentar a IA, incluindo origem, formato e quantidade mínima necessária.
- Criar um diagrama simples mostrando como a IA vai se comunicar com o APEX e com o Oracle Database.
- Descrever o fluxo de funcionamento da IA dentro do sistema, explicando o que acontece quando o usuário aciona essa funcionalidade no APEX.

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Critérios de Avaliação

(até 60 pontos) Aplicação técnica de conceitos de IA.

(até 20 pontos) Clareza e didática da apresentação em vídeo

(**até 20 pontos)** Organização do repositório e documentação técnica.

Entregáveis obrigatórios:

Arquivo .zip contendo:

- Link do vídeo publicado no YouTube (em modo não listado) com a apresentação da ideia central do projeto, demonstração funcional (real ou simulada) dos principais recursos implementados.
- Link para o repositório no GitHub, apresentando código-fonte e estrutura do projeto e o README com instruções de uso, tecnologias utilizadas e resultados parciais.

Condições de entrega

- A integridade e o conteúdo do arquivo entregue são de responsabilidade dos integrantes do grupo.
- Arquivos entregues sem conteúdo ou com arquivos corrompidos não serão considerados. Sugestão: confira seu anexo antes de publicar.
- Não serão aceitos arquivos enviados pelo Teams ou fora do prazo.

JAVA ADVANCED 1/3

Você deverá desenvolver uma aplicação web completa utilizando o framework Spring Boot para dar suporte à solução proposta. Esta aplicação deve ter o foco nos seguintes tópicos:

1. Camada de visualização (frontend);
2. Flyway para controle de versões do banco de dados;
3. Spring Security para autenticação e controle de acesso.

Requisitos

A aplicação deverá conter obrigatoriamente os seguintes elementos técnicos:

1. Frontend (30 pontos)
2. Flyway (20 pontos)
3. Spring Security (30 pontos)
   - Implementação de pelo menos dois tipos de usuário, com permissões diferentes.
   - Proteção de rotas com base no perfil do usuário.
4. Funcionalidades completas (20 pontos)
   - Implementação funcional de pelo menos dois fluxos completos do sistema (exceto CRUD)
   - Validações básicas nos formulários e nos dados.

JAVA ADVANCED 2/3

Penalidades

As penalidades abaixo serão aplicadas independentemente dos requisitos técnicos, e dizem respeito à qualidade do código e funcionamento geral do
sistema:

Problema	Desconto
Violação evidente de princípios SOLID (ex: métodos gigantes, responsabilidades múltiplas)	-10 pontos por ocorrência
Código com repetições desnecessárias (violação de DRY)	-5 pontos por ocorrência
Código com problemas de legibilidade (violação de Clean Code	-5 pontos por ocorrência
Comentários no lugar de refatorações claras	-3 pontos
Funcionalidade com comportamento inesperado ou erro evidente	-5 pontos por funcionalidade
Página que não carrega ou link quebrado	-5 pontos por página/link
Apresentação incompleta ou incoerente com a proposta do challenge.	-15 pontos

JAVA ADVANCED 3/3

Entrega
Os seguintes artefatos devem ser entregues no portal até o prazo definido. Não serão aceitas entregas após o prazo ou por outros meios.
• Link do repositório público do GitHub com o código fonte da aplicação Spring completa.
• O readme do repositório deve conter todas as instruções necessárias para a instalação, execução e acesso da aplicação.
• Vídeo demonstração da aplicação funcionando com apresentação das principais funcionalidades da aplicação web (máx. 10 min.)

Avaliação Oral
Após a entrega do projeto, cada aluno participará de uma avaliação oral individual em sala de aula. Essa etapa complementa a avaliação técnica. O objetivo é verificar a compreensão do código entregue e promover o uso consciente de ferramentas como a inteligência artificial. Durante essa conversa, o aluno deverá:
• Explicar trechos específicos do próprio código;
• Justificar decisões de implementação;
• Comentar eventuais dificuldades encontradas;
• Descrever se e como utilizou ferramentas de IA no processo

MOBILE APP DEVELOPMENT 1/11

Esta é a entrega intermediária mais importante do projeto. Ao final desta sprint, o aplicativo **deve estar funcional**, permitindo que o problema escolhido pelo grupo seja efetivamente resolvido, ainda que de forma parcial.

O protótipo entregue na Sprint 1 **não é mais aceitável como referência visual apenas**: ele deve ter sido convertido em uma base real de desenvolvimento. A aplicação deve conter fluxos principais implementados, integração consistente com a API definida na Sprint 2 e uma interface utilizável, ainda que não finalizada.

Nesta etapa, espera-se que:

- As funcionalidades centrais do sistema estejam implementadas
- O app execute ações reais (criar, consultar, atualizar ou processar dados)
- A integração com a API funcione de ponta a ponta
- O código esteja organizado, legível e compatível com boas práticas
- A aplicação não seja apenas uma navegação de telas sem lógica real

A Sprint 3 **não é fase de rascunho**, **protótipo navegável ou simulação**. Ela representa a consolidação técnica do projeto, que será refinado e finalizado na Sprint 4.

MOBILE APP DEVELOPMENT 2/11

Requisitos Avaliativos

- Navegação entre telas (5 pontos)
- Integração com API backend (HTTP) (15 pontos)
- Sistema de autenticação (Login) (20 pontos)
- Funcionalidade integrada com Oracle APEX (20 pontos)
- Arquitetura e organização do código (20 pontos)
- Documentação e apresentação da entrega (20 pontos)

Total: 100 pontos

Navegação entre Telas (5 pontos)

A aplicação deve implementar navegação funcional entre telas, utilizando uma biblioteca de navegação apropriada e configuração explícita de rotas.

Critérios de Avaliação

- **Quantidade mínima de telas** (2 pontos)
  - O aplicativo deve possuir **no mínimo 6 telas distintas**
  - Telas duplicadas, telas vazias ou variações visuais da mesma tela **não serão contabilizadas**
  - Cada tela deve representar uma funcionalidade ou fluxo diferente do sistema

- **Uso de biblioteca de navegação** (2 pontos)
  - A navegação deve ser implementada utilizando uma **biblioteca de navegação** (ex: React Navigation ou expo router)
  - Não será aceito controle manual de telas via condicionais, if, useState ou renderização condicional como substituto de navegação

- **Configuração correta das rotas** (1 ponto)
  - As rotas devem estar **explicitamente declaradas**
  - A navegação entre telas deve ocorrer por meio dessas rotas
  - Navegação quebrada, rotas inexistentes ou não utilizadas implicam perda deste ponto

Integração com API Backend HTTP (15 pontos)

A aplicação deve integrar-se a um backend acessado via HTTP, realizando operações reais de comunicação com a API definida pelo grupo.

Critérios de Avaliação

- **Integração real com API HTTP** (5 pontos)
  - As requisições devem ser implementadas com **TanStack Query** (useQuery, useMutation ou equivalentes)
  - Os dados exibidos na interface devem ser **provenientes exclusivamente da API**
  - Não será aceito uso de dados mockados, arquivos locais ou valores fixos no código
- **Implementação de funcionalidades baseadas em API** (4 pontos)
  - O aplicativo deve possuir **no mínimo duas funcionalidades distintas** dependentes da API
  - Cada funcionalidade deve utilizar dados reais retornados pelo backend e refletir alterações feitas pelo usuário
- **CRUD completo integrado à interface** (4 pontos)
  - As operações de **Create, Read, Update e Delete** devem estar implementadas nas duas funcionalidades
  - Todas as operações devem ser acessíveis por meio da interface do aplicativo
  - Não será aceito CRUD parcialmente implementado ou simulado
- **Estados de carregamento e atualização de dados** (2 pontos)
  - A aplicação deve exibir **estado de carregamento (loading)** durante requisições
  - Alterações nos dados (create/update/delete) devem refletir **automaticamente na interface**, sem necessidade de reiniciar o app ou recarregar manualmente

MOBILE APP DEVELOPMENT 5/11

Funcionalidade Integrada com Oracle APEX (20 pontos)

A aplicação deve integrar-se a uma funcionalidade implementada no Oracle APEX, sendo esta integração parte essencial da solução proposta pelo grupo.

Critérios de Avaliação

- **Funcionalidade real implementada no Oracle APEX** (6 pontos)
  - O grupo deve implementar uma funcionalidade real utilizando o Oracle APEX
  - A funcionalidade deve ir além de telas de cadastro simples ou exemplos genéricos
  - A lógica principal da funcionalidade deve estar no APEX (ex: regras de negócio, validações ou processamento de dados)

- **Exposição da funcionalidade via API no APEX** (6 pontos)
  - A funcionalidade desenvolvida no APEX deve ser exposta por meio de API REST
  - O endpoint deve permitir interação real com os dados ou processos definidos no APEX
  - Endpoints apenas de teste ou demonstração não serão aceitos

- **Consumo da API do APEX pelo aplicativo** (6 pontos)
  - O aplicativo deve consumir a API exposta pelo APEX para executar a funcionalidade
  - A interação com o APEX deve ser essencial para o funcionamento do app
  - A aplicação não deve funcionar corretamente caso a API do APEX esteja indisponível

- **Integração visível e demonstrável** (2 pontos)
  - Durante a apresentação, deve ser possível demonstrar claramente:
    - A funcionalidade no APEX
    - A chamada à API
    - O reflexo do resultado no aplicativo

Arquitetura e Organização do Código (20 pontos)

O projeto deve apresentar uma arquitetura clara, organizada e coerente, adequada a uma aplicação real em produção.

Critérios de Avaliação

- **Separação de responsabilidades** (6 pontos)
  - O código deve estar organizado separando claramente:
    - Interface (telas/componentes)
    - Lógica de negócio
    - Camada de acesso a dados / APIs
  - Não será aceito código de regra de negócio ou chamadas HTTP diretamente em componentes de tela
- **Estrutura de pastas consistente** (5 pontos)
  - O projeto deve possuir uma estrutura de pastas clara e padronizada
  - Arquivos não devem estar todos concentrados em uma única pasta
  - A organização deve permitir identificar facilmente onde estão telas, serviços, hooks e componentes reutilizáveis
- **Uso adequado de hooks e abstrações** (5 pontos)
  - Lógica reutilizável deve estar abstraída em hooks ou serviços
  - Não será aceito código duplicado em múltiplas telas sem justificativa
  - Hooks como os do TanStack Query devem estar isolados da camada de UI
- **Legibilidade e manutenibilidade do código** (4 pontos)
  - Nomes de arquivos, funções e variáveis devem ser coerentes e autoexplicativos
  - Código excessivamente acoplado, confuso ou difícil de entender implicará perda de pontuação
  - O projeto deve permitir evolução sem necessidade de reescrita completa

Documentação e Apresentação da Entrega (20 pontos)

A entrega deve conter documentação escrita e apresentação em vídeo, demonstrando claramente o funcionamento do aplicativo e o atendimento aos requisitos da Sprint 3.

Critérios de Avaliação

- **Documentação no README do repositório** (5 pontos)
  - O projeto deve possuir um arquivo **README.md**, contendo no mínimo:
    - Descrição do problema escolhido
    - Descrição geral da solução proposta
    - Tecnologias utilizadas
    - Instruções básicas para execução do projeto
- **Vídeo de apresentação do aplicativo** (15 pontos)
  - O grupo deve gravar um **vídeo com narração**, demonstrando o aplicativo em funcionamento
  - O vídeo deve ser **publicado no YouTube** e o link informado junto à entrega, no README.md
  - O vídeo deve apresentar, de forma clara e objetiva:
    - Navegação entre as telas
    - Sistema de autenticação
    - Integração com API backend
    - Funcionalidade integrada com Oracle APEX
    - Comportamento do aplicativo em uso real
  - A demonstração deve evidenciar explicitamente os **requisitos avaliativos da Sprint 3**
- **Duração e formato do vídeo**
  - Duração máxima: **5 minutos**
  - Vídeos sem narração, com áudio inaudível ou apenas com slides **não serão aceitos**
  - Vídeos que não demonstrem o funcionamento real do app terão pontuação reduzida ou zerada

MOBILE APP DEVELOPMENT 8/11

Penalidades

As penalidades abaixo serão aplicadas independentemente da pontuação obtida nos requisitos avaliativos, podendo resultar em redução significativa da nota final.

I. **Não realizar a entrega via GitHub Classroom** (-20 pontos)

A entrega deve ser feita exclusivamente pelo repositório oficial do GitHub Classroom.

II. **Ausência de vídeo de apresentação** (-20 pontos)

A não entrega do vídeo ou entrega em formato diferente do especificado implica penalização integral deste item.

III. **Ausência ou inadequação do arquivo README.md** (-10 pontos)

Projetos sem README ou com documentação claramente insuficiente estarão sujeitos à penalização.

IV. **Não utilização de tema (modo claro e/ou escuro)** (-20 pontos)

O aplicativo deve implementar suporte a tema. Aplicações sem qualquer tratamento de tema serão penalizadas.

V. **Aplicativo fora do escopo das aulas** (-60 pontos)

Aplicações que utilizem soluções, frameworks, arquiteturas ou abordagens não vistas ou não autorizadas em aula, de forma a **simplificar artificialmente o desenvolvimento**, sofrerão penalização severa.

VI. **Remoção de telas ou funcionalidades entregues em sprints anteriores** (-100 pontos)

Não é permitido remover, desativar ou inutilizar telas e funcionalidades já entregues e avaliadas anteriormente. Esta penalidade implica **nota zero na Sprint**.

VII. **Histórico de commits incoerente, inexistente ou artificial** (-50 pontos)

É esperado que o repositório apresente:

- Evolução gradual do projeto
- Commits frequentes e coerentes
- Mensagens claras e descritivas

Repositórios com histórico artificial (ex: poucos commits grandes, commits genéricos, ou upload único do projeto) ou que não demonstrem envolvimento prático com o desenvolvimento poderão ser penalizados.

MOBILE APP DEVELOPMENT 9/11

Penalidades

VIII. Integrações simuladas ou incompletas (-40 pontos)

Aplica-se quando for identificado qualquer um dos seguintes casos:

- Uso de API apenas para operações de leitura (GET)
- CRUD incompleto ou parcialmente simulado
- Código de integração não utilizado pela interface
- Dados retornados pela API ignorados ou substituídos por valores fixos ou mockados
- Necessidade de reiniciar o aplicativo para refletir alterações de dados
- Atualização manual da interface com useState após requisições

Esta penalidade cobre **todas as tentativas de "fingir integração"**.

IX. Navegação simulada ou inexistente (-30 pontos)

Aplica-se quando a navegação do aplicativo não for baseada em rotas reais.

Inclui, mas não se limita a:

- Alteração de componentes dentro da mesma tela como substituto de navegação
- Exibição/ocultação de interface sem troca de rota
- Uso de modais para simular telas
- Uso de dados estáticos ou mockados como substituto de fluxos reais

X. Sistema de autenticação fictício ou incompleto (-40 pontos)

Aplica-se quando o sistema de autenticação não for real ou puder ser facilmente burlado.

Inclui:

- Login com usuário e senha fixos no código
- Controle de acesso feito apenas com variáveis locais ou useState
- Telas protegidas acessíveis sem autenticação
- Persistência de sessão inexistente ou simulada

MOBILE APP DEVELOPMENT 10/11

Penalidades

XI. Uso inadequado ou irrelevante do Oracle APEX (-50 pontos)

Aplica-se quando o Oracle APEX não for parte essencial da solução.

Inclui:

- Uso do APEX apenas como banco de dados
- Uso do APEX apenas para cadastros simples sem lógica associada
- API criada no APEX que não é utilizada pelo aplicativo
- Funcionalidade que poderia existir integralmente sem o APEX

Esta penalidade pode ser aplicada **independentemente da existência de API**.

XII. Arquitetura inadequada ou código não manutenível (-30 pontos)

Aplica-se quando o projeto apresentar problemas estruturais graves, tais como:

- Toda a lógica concentrada em componentes de tela
- Um único arquivo contendo múltiplas responsabilidades
- Repetição excessiva de código sem abstração
- Estrutura que dificulte a leitura, manutenção ou evolução do projeto

XIII. Aplicativo não funcional (-100 pontos)

Será atribuída **nota zero na Sprint** caso o aplicativo:

- Não execute corretamente
- Não inicie (erro de build, crash ao abrir)
- Apresente falhas que impeçam a demonstração das funcionalidades avaliativas
- Não permita a execução dos fluxos principais do sistema durante a avaliação
- Dependa de ajustes manuais, configurações externas não documentadas ou intervenção do professor para funcionar

Penalidades

XIV. Vídeo de apresentação incompatível com o aplicativo entregue (-100 pontos)

Será atribuída **nota zero na Sprint** caso o vídeo de apresentação:

- Não demonstre o **aplicativo real entregue no repositório**
- Apresente protótipos, maquetes ou telas de ferramentas como **Figma, Adobe XD ou similares**
- Demonstre uma versão diferente da aplicação entregue
- Não permita verificar, de forma inequívoca, que o vídeo corresponde ao código avaliado

O vídeo deve demonstrar **exclusivamente o aplicativo funcional**, em execução, conforme entregue para avaliação.

Observações Gerais

- As penalidades **são cumulativas**
- A aplicação de uma penalidade **não exige a ocorrência de todos os itens listados**
- A identificação de **um único caso representativo** é suficiente para aplicação
- Penalidades podem resultar em **nota final zero**, independentemente da pontuação obtida nos critérios avaliativos

Sprint 4

COMPLIANCE & QUALITY ASSURANCE

ITENS OBRIGATÓRIOS:

PARTE A)

Crie o plano de testes manuais, de validação no nível de sistema para seu projeto (as funcionalidades principais do seu sistema precisam estar
cobertas), usando a plataforma AZURE BOARDS:

1)Liste os testes planejados – peso 20%

2)Descreva os dados de entrada para cada teste – peso 20%

3)Descreva os dados de saída para cada teste – peso 20%

4)Descreva o procedimento (passos) de teste aplicado – peso 20%

Obs: trabalhe com dados controlados nos seus testes, predefinindo os valores de variáveis para input e output esperado. Os testes precisam estar de acordo com o plano de release e tarefas de sprints realizadas. Esses itens serão checados e em caso de desrespeito às regras, será aplicada uma penalidade de 10% na nota.

PARTE B)

Utilize um software de automação para validar o seu sistema. Pode ser usada automação Record & Playback com Selenium IDE ou Katalon Studio se o seu sistema tiver telas de interface de usuário. Se o seu sistema é composto somente por APIs e componentes de serviços sem interface de usuário, empregue POSTMAN ou software equivalente na automação:

Faça ao menos 4 casos de testes automatizados para a sua aplicação – peso (20%)

FORMA DE ENTREGA:

• Crie um repositório no GIT HUB, com acesso público e suba na Branch develop: a) Link de acesso ao AZURE BOARDS para correção dos testes manuais, sendo que o professor precisa ser membro da organização e do projeto para poder acessar e corrigir o trabalho;. B) Link de vídeo que mostra a configuração e execução dos testes de automação, nas ferramentas escolhidas.

• Entregas fora do formato implicarão em penalidade de 10% na nota.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

MongoDB - Migração do seu projeto Sprint 3 para NoSql:

❑ Projeto de Banco de Dados NoSQL MongoDB (até 15 pontos):
❑ Documentação do Projeto:
- Apresente uma descrição detalhada do projeto e a justificativa para a escolha do MongoDB como Banco de Dados NoSQL.
❑ Modelo de Dados e Justificativas:
- Descreva os modelos de dados utilizados, explicando como as estruturas de dados atendem aos requisitos do projeto.

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

MongoDB:

❑ Construção de Dados e Operações (até 20 pontos):
- Realize as seguintes tarefas com base no MongoDB:
- Criação de Documentos JSON/BSON:
    - Crie 10 documentos em formato JSON/BSON, cada um com no mínimo 10 atributos.

❑ Interface de Consulta de Dados (30 pontos):
- Implemente uma interface que permita:
- Consulta de Dados: Visualização e busca de documentos no MongoDB.
- Operações CRUD (Create, Read, Update, Delete): Interface para inserir, atualizar, e excluir documentos. ❑

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

MongoDB:

- Exportação de Dataset (20 pontos):
- Exporte um dataset dos dados inseridos no MongoDB, demonstrando os documentos criados e sua estrutura.

- Entregável: Arquivo contendo o PDF do Projeto de Banco de Dados NoSQL MongoDB e os arquivos de script referentes ao banco de dados escolhido.

- Vídeo com aúdio demonstrando o funcionamento da estrutura de da exexução de codificação. (15 pontos)

MASTERING RELATIONAL AND NON RELATIONAL DATABASE

Penalidades:

- A falta da documentação compromete totalmente a correção das instruções em NOSQL, acarretando uma nota mínima de entrega de 30% do total da atividade.
- O não cumprimento dos requisitos solicitados na codificação implica na perda de 50% do total da nota do item.
- A criação de uma ou mais instruções que não atendam o solicitado será desconsiderada, sem valor na pontuação.
- A instrução que atenda parcialmente o solicitado terá como valor 50% do total deste item.
- Instruções que apresentarem problemas de execução serão desconsideradas e não haverá pontuação para a mesma.
- Video de baixa resolução, sem áudio e faltando explicação será desconsiderado.

DEVOPS TOOLS E CLOUD COMPUTING 1/2

Utilizando o conhecimento adquirido em DevOps, é esperado o grupo inserir uma solução ou parte dela (um módulo) em uma típica esteira contínua DevOps (Pipeline), permitindo realizar Deploy de forma mais rápida e com menos custo operacional, melhorando significativamente a qualidade das aplicações:

Entregas obrigatórias:

1.Uma breve descrição de sua solução; (0 pontos – A falta desse quesito será descontado 05 pontos);

2.O desenho da Pipeline de integração contínua e entrega contínua (CI/CD) com foco na solução definida pelo seu grupo. Inclua o detalhamento de cada etapa criada (dissertação); **(20 pontos);**

3.Configuração das pipelines (CI e CD) definido na ferramenta Azure DevOps; (40 pontos);

4.Uma gravação de um vídeo mostrando o funcionamento da solução, desde a entrada no AzureDevOps, executando a Pipeline e mostrando o funcionamento da solução até persistência de dados em Nuvem. Mostrar os dados persistidos no banco inclusive; **(30 pontos);**

DEVOPS TOOLS E CLOUD COMPUTING 2/2

5.Código fonte da aplicação no Github, código da Pipeline (e com tudo que é necessário para a execução) com um Read.me para realizar a execução da Pipeline e testes (o Prof irá reproduzir e testar com base nesse Readme.md). Incluir os scripts JSON do CRUD se a solução for uma API; (até 05 pontos);

6.Deve-se utilizar pelo menos duas tabelas com um relacionamento entre elas e o Banco de Dados em Nuvem; **(Até 05 pontos);**

7.Arquivo PDF, contendo o nome e RM dos integrantes do time, link do Github e link do YouTube.

A PONTUAÇÃO DE CADA QUESITO VAI VARIAR DE ACORDO COM A QUALIDADE DA TAREFA ENTREGUE

ADVANCED BUSINESS DEVELOPMENT WITH .NET

Você deverá entregar o projeto completo e funcional, contemplando todas as funcionalidades implementadas ao longo das Sprints 1, 2 e 3, integradas e funcionando em conjunto. Esta sprint tem foco na consolidação de todos os tópicos:

1. Arquitetura e qualidade de código;
2. API Restful completa;
3. Persistência de dados relacional e NoSQL;
4. Monitoramento, observabilidade e testes;
5. Documentação final.

Requisitos

1. Arquitetura e Código (30 pts)

- Clean Architecture aplicada corretamente com separação de camadas.
- Princípios SOLID e Clean Code evidentes no código.
- Injeção de Dependência configurada adequadamente.
- Tratamento de exceções global e consistente.

ADVANCED BUSINESS DEVELOPMENT WITH .NET

2. API Restful Completa (20 pts)

- Documentação Swagger/OpenAPI atualizada.
- Paginação, ordenação e filtros implementados.
- HATEOAS implementado nos endpoints de consulta.
- Autenticação e Autorização com JWT ou Identity.

3. Persistência de Dados (20 pts)

- Entity Framework Core com migrações aplicadas (Oracle ou SQL Server).
- MongoDB integrado para operações NoSQL.
- Repositórios concretos implementados com padrão Repository.

4. Monitoramento, Observabilidade e Testes (20 pts)

- Health Checks configurados e funcionais.
- Logging estruturado implementado.
- Testes unitários e de integração com padrão AAA (xUnit).
- Cobertura mínima de testes das camadas de Domínio e Aplicação.

5. Documentação e README Final (10 pts)

- README completo: visão geral, arquitetura, endpoints, instalação, testes e integrantes.
- Diagrama de arquitetura da solução.
- Documentação dos endpoints via Swagger exportado ou descrito no README.

Penalidades:

-20 pts (Projeto não compilar)

-10 pts (Ausência de README ou README incompleto)

-30 pts (Ausência total de testes)

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Objetivos Principais

- Implementar o modelo de Inteligência Artificial definido na 3ª entrega, integrá-lo à aplicação Oracle APEX e demonstrar seu funcionamento real por meio de testes, documentação técnica e vídeo-pitch.
- Gravar um vídeo pitch de aproximadamente 5 minutos com o modelo de IA e integração com o APEX funcionando.

Objetivos Específicos

- Desenvolver o modelo de IA conforme especificado na entrega anterior.
- Construir o serviço de IA consumível pela aplicação APEX.
- Integrar a IA com a aplicação Oracle APEX.
- Testar o funcionamento da IA no sistema.

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Tarefas

- Implementar o modelo de IA definido na primeira entrega (treinamento, ajuste ou configuração do modelo escolhido).
- Preparar os dados necessários para o funcionamento da IA (pré-processamento, organização e testes).
- Criar o serviço ou mecanismo que permitirá que o APEX envie dados para a IA e receba os resultados (API, REST, script, função etc.).
- Integrar o modelo de IA à aplicação Oracle APEX, garantindo que a funcionalidade esteja acessível ao usuário final.
- Testar o funcionamento da IA dentro do sistema, registrando entradas, saídas e evidências de execução.
- Documentar o funcionamento da solução de IA, incluindo explicação do modelo, fluxo de integração e instruções de uso.

DISRUPTIVE ARCHITECTURES: IOT, IOB & GENERATIVE IA

Critérios de Avaliação:

- (até 60 pontos) Aplicação técnica de conceitos de IA.
- (até 20 pontos) Clareza e didática da apresentação em vídeo
- (até 20 pontos) Organização do repositório e documentação técnica.

Entregáveis obrigatórios:

Arquivo .zip contendo:

- Link do vídeo publicado no YouTube (em modo não listado) com a apresentação da ideia central do projeto, demonstração
- funcional (real ou simulada) dos principais recursos implementados.
- Link para o repositório no GitHub, apresentando código-fonte e estrutura do projeto e o README com instruções de uso, tecnologias utilizadas e resultados parciais.

Condições de entrega:

- A integridade e o conteúdo do arquivo entregue são de responsabilidade dos integrantes do grupo.
- Arquivos entregues sem conteúdo ou com arquivos corrompidos não serão considerados. Sugestão: confira seu anexo antes de publicar. Não serão aceitos arquivos enviados pelo Teams ou fora do prazo.

JAVA ADVANCED 1/4

Durante o semestre, seu grupo trabalhou em um projeto cujo desafio central foi propor e implementar uma solução tecnológica inovadora para um problema real. Agora é o momento de consolidar todo esse trabalho e apresentar o resultado final de forma clara, funcional e integrada, demonstrando:

- O funcionamento do sistema;
- A coerência da solução proposta com o desafio;
- O uso das tecnologias aprendidas;
- E a conexão com as demais disciplinas do semestre.

Requisitos da Entrega Final

1. Demonstração Técnica da Solução (40 pontos)

- A aplicação deve estar rodando online (deploy)
- A equipe deve navegar pelos principais fluxos do sistema.
- A solução deve aplicar os principais conceitos da disciplina de forma contextualizada com o app criado.
- A interface deve apresentar boa UI e UX

JAVA ADVANCED 2/4

2. Narrativa da Solução (20 pontos)
• Explicação clara da proposta da solução do grupo.
• Apresentação das decisões de design, escolhas tecnológicas e justificativas.
• Destaque para a originalidade e criatividade da solução.

3. Integração Multidisciplinar (20 pontos)
• Explicitação e demonstração de como as demais disciplinas foram aplicadas na solução.
• Apresentação de evidências: documentação, canvas, protótipos, scripts SQL, etc.

4. Apresentação Oral e Comunicação em Equipe (10 pontos)
• Todos os membros devem participar do vídeo.
• Clareza, objetividade e domínio sobre o que está sendo demonstrado.

5. Organização da entrega e da documentação (10 pontos)

JAVA ADVANCED 3/4

Penalidades

Problema	Desconto
Código com violações evidentes de princípios de boas práticas	-10 pontos por ocorrência
Repetição de código que poderia ter sido extraído para métodos reutilizáveis ou templates	-5 pontos por ocorrência
Ausência de evidência de colaboração entre membros	-10 pontos
Falhas graves de usabilidade na interface	-5 pontos por ocorrência
Erros visuais ou de fluxo lógico durante a apresentação	-5 pontos
Falta de alinhamento com o problema proposto (ex: solução genérica que ignora o desafio proposto)	-10 pontos
Entrega fora do prazo ou fora do portal	-100 pontos

JAVA ADVANCED 4/4

Artefatos de Entrega

- Repositório no GitHub com README completo.
- Link de acesso à aplicação.
- Vídeo da apresentação com duração máxima de 15 minutos.

MOBILE APP DEVELOPMENT 1/11

Esta é a **entrega final do projeto** e representa o fechamento do trabalho integrado da disciplina. O aplicativo mobile desenvolvido em **React Native** deve ser apresentado em sua **versão completa, funcional e finalizada**, refletindo todo o processo evolutivo construído ao longo das sprints anteriores e os feedbacks recebidos.

Espera-se um **produto pronto para uso**, no qual todas as funcionalidades planejadas estejam implementadas, integradas e funcionando de forma consistente. A aplicação deve demonstrar maturidade técnica, estabilidade, atenção aos detalhes e domínio das tecnologias utilizadas, especialmente a integração entre **Oracle APEX** e o aplicativo mobile.

Nesta etapa, o foco não é mais prototipação ou experimentação, mas sim a **entrega de uma solução real**, capaz de ser utilizada por usuários finais. Esta sprint avalia a capacidade do grupo de **concluir, consolidar e entregar** um produto mobile de qualidade, compatível com o nível esperado de conclusão da graduação.

MOBILE APP DEVELOPMENT 2/11

Requisitos Avaliativos

- **Implementação funcional de todas as telas do aplicativo** (20 pontos)
- **Integração final e completa com Oracle APEX** (20 pontos)
- **Publicação do aplicativo** (10 pontos)
- **Notificações (Push ou Local)** (20 pontos)
- **Arquitetura de código (React Native)** (10 pontos)
- **Vídeo de Apresentação e Demonstração do Aplicativo** (20 pontos)

Total: 100 pontos

Implementação Funcional de Todas as Telas do Aplicativo (20 pontos)

Este requisito avalia se o aplicativo entregue na Sprint 4 representa a **conclusão funcional do trabalho desenvolvido ao longo das três sprints anteriores**, resultando em uma **solução utilizável e coerente** para o problema proposto.

Critérios de Avaliação

- **Funcionalidade real de todas as telas** (6 pontos)
  - Todas as telas implementadas devem possuir comportamento funcional
  - Telas apenas visuais, demonstrativas ou sem efeito prático implicam perda proporcional ou total desta pontuação
- **Fluxo completo e contínuo de uso** (6 pontos)
  - O aplicativo deve permitir que o usuário execute o **fluxo principal da solução do início ao fim**
  - Fluxos interrompidos, etapas inoperantes ou telas sem ação resultam em perda desta pontuação
- **Conclusão das funcionalidades das sprints anteriores** (4 pontos)
  - Funcionalidades apresentadas nas Sprints 1, 2 e 3 devem estar **finalizadas e integradas**
  - Funcionalidades parciais, simuladas ou abandonadas implicam perda desta pontuação
- **Utilidade prática da solução** (4 pontos)
  - A aplicação deve entregar **valor prático mínimo** ao usuário final
  - Aplicativos que cumpram requisitos técnicos, mas sejam inutilizáveis na prática, poderão perder esta pontuação

Integração Final e Completa com Oracle APEX (20 pontos)

Este requisito avalia se o **Oracle APEX é parte essencial e determinante da solução final**, sendo responsável pelo processamento dos dados e pela geração das informações que resolvem o problema proposto, com **resultado claramente visível no aplicativo mobile**.

Critérios de Avaliação

- **Processamento de dados e lógica no APEX** (6 pontos)
  - O Oracle APEX deve executar processamento relevante dos dados
  - A solução deve gerar **informações derivadas**, e não apenas armazenar ou repassar dados brutos
  - Regras de negócio, cálculos, consolidações ou validações devem ocorrer no APEX

- **Resultado da funcionalidade visível no aplicativo** (6 pontos)
  - O aplicativo deve apresentar claramente o **resultado produzido pelo APEX**
  - O usuário deve conseguir identificar que a informação exibida é consequência direta do processamento realizado no APEX
  - Resultados não visíveis ou não compreensíveis implicam perda desta pontuação

- **Integração estável e consistente** (4 pontos)
  - A comunicação entre o aplicativo e a API do APEX deve ocorrer de forma estável
  - O aplicativo deve tratar corretamente respostas de sucesso e erro
  - Falhas de integração que impeçam o uso normal do app implicam perda desta pontuação

- **Dependência real do APEX para funcionamento da solução** (4 pontos)
  - A funcionalidade central do aplicativo deve **depender do APEX para existir**
  - Soluções que poderiam funcionar sem o APEX ou com lógica equivalente no app perderão esta pontuação

Publicação do Aplicativo (10 pontos)

Este requisito avalia se o aplicativo foi **efetivamente publicado** e se a versão disponibilizada corresponde **exatamente ao código-fonte entregue para avaliação**.

Critérios de Avaliação

- **Publicação no Firebase App Distribution** (4 pontos)
  - O aplicativo deve estar publicado no **Firebase App Distribution**
  - A versão deve estar disponível para instalação e execução
- **Acesso do professor como tester**
  - O e-mail institucional do professor deve estar cadastrado como **tester**
  - O acesso deve estar funcional no momento da avaliação
- **Correspondência entre aplicativo publicado e código entregue** (3 pontos)
  - A versão publicada deve corresponder **exatamente** ao código-fonte enviado no repositório
  - Diferenças entre a versão publicada e o código avaliado implicam perda desta pontuação
- **Tela "Sobre o App" com identificação da versão** (3 pontos)
  - O aplicativo deve possuir uma tela **"Sobre o App"**
  - Esta tela deve exibir claramente o **hash do commit de referência** da versão publicada

Notificações (Push ou Local) (20 pontos)

Este requisito avalia a implementação de **notificações reais**, podendo ser do tipo **push** ou **local**, integradas ao contexto da aplicação e relevantes para a solução proposta.

Critérios de Avaliação

- **Cenário realista de notificação** (6 pontos)
  - O aplicativo deve implementar **ao menos um cenário realista** de notificação
  - A notificação deve estar diretamente relacionada ao contexto e às funcionalidades do aplicativo
  - Notificações genéricas ou desconectadas da proposta da solução implicam perda desta pontuação
- **Disparo e recepção funcionais** (6 pontos)
  - A notificação deve ser disparada a partir de um evento real do sistema (ex: ação do usuário, processamento no backend, evento no APEX ou serviço externo)
  - No caso de **push**, deve ser recebida corretamente com o app em foreground ou background
  - No caso de **local**, deve ser exibida automaticamente durante o uso do aplicativo
- **Integração com o fluxo do aplicativo** (4 pontos)
  - A notificação deve conduzir o usuário a uma tela, ação ou informação relevante
  - A interação com a notificação deve ter impacto real no uso do app
- **Demonstração no vídeo de apresentação** (4 pontos)
  - O funcionamento da notificação deve ser claramente demonstrado no vídeo
  - Deve ser possível identificar o evento gerador e o comportamento do aplicativo

Observação

A escolha entre notificação **push** ou **local** é livre, desde que a notificação seja funcional, relevante para o contexto da aplicação e represente um uso realista em um produto mobile. Alertas simulados, mensagens fixas, notificações disparadas manualmente apenas para demonstração ou que não tenham impacto real no fluxo de uso do aplicativo não serão consideradas válidas para fins de avaliação.

Arquitetura e Organização do Código (10 pontos)

O projeto deve apresentar uma arquitetura clara, organizada e coerente, adequada a uma aplicação real em produção.

Critérios de Avaliação

- **Separação de responsabilidades** (4 pontos)
  - O código deve estar organizado separando claramente:
    - Interface (telas/componentes)
    - Lógica de negócio
    - Camada de acesso a dados / APIs
  - Não será aceito código de regra de negócio ou chamadas HTTP diretamente em componentes de tela
- **Estrutura de pastas consistente** (3 pontos)
  - O projeto deve possuir uma estrutura de pastas clara e padronizada
  - Arquivos não devem estar todos concentrados em uma única pasta
  - A organização deve permitir identificar facilmente onde estão telas, serviços, hooks e componentes reutilizáveis
- **Uso adequado de hooks e abstrações** (2 pontos)
  - Lógica reutilizável deve estar abstraída em hooks ou serviços
  - Não será aceito código duplicado em múltiplas telas sem justificativa
  - Hooks como os do TanStack Query devem estar isolados da camada de UI
- **Legibilidade e manutenibilidade do código** (1 pontos)
  - Nomes de arquivos, funções e variáveis devem ser coerentes e autoexplicativos
  - Código excessivamente acoplado, confuso ou difícil de entender implicará perda de pontuação

Observação

Como este critério já foi avaliado na Sprint 3, espera-se que a arquitetura esteja **mantida ou aprimorada**. Regressões arquiteturais em relação à entrega anterior poderão resultar em perda total ou parcial da pontuação.

Vídeo de Apresentação e Demonstração do Aplicativo (20 pontos)

O vídeo deve apresentar de forma clara, objetiva e contínua o resultado final do aplicativo, demonstrando todas as funcionalidades propostas ao longo do projeto.
Este item representa a **conclusão do trabalho desenvolvido nas três sprints anteriores** e deve evidenciar que o aplicativo está funcional, integrado e pronto para uso.

Critérios de Avaliação

- **Demonstração completa das funcionalidades propostas (8 pontos)**
  - Todas as funcionalidades principais definidas para o projeto devem ser demonstradas em funcionamento.
  - Cada funcionalidade deve ser apresentada do início ao fim do fluxo (entrada → processamento → resultado).
  - Funcionalidades não demonstradas no vídeo serão consideradas **não implementadas**, mesmo que existam no código.

  Perda total dos pontos deste subitem caso o vídeo não cubra as funcionalidades essenciais do aplicativo.

- **Demonstração do fluxo real de uso do aplicativo (6 pontos)**
  - O vídeo deve mostrar o uso do aplicativo do ponto de vista do usuário final.
  - O fluxo deve ser contínuo, sem cortes que ocultem erros, falhas ou comportamentos inconsistentes.
  - O aplicativo deve permitir concluir ao menos um cenário completo de uso real, sem bloqueios.

  Aplicativos que "funcionam por telas isoladas", mas não permitem um uso real e contínuo, perdem a pontuação deste subitem.

- **Coerência entre vídeo, código e versão publicada (4 pontos)**
  - O aplicativo demonstrado no vídeo deve corresponder exatamente ao código entregue no repositório.
  - A versão exibida no vídeo deve ser a mesma publicada e disponibilizada ao professor.
  - Divergências entre vídeo, código e versão publicada implicam perda total da pontuação deste subitem.

- **Clareza técnica da demonstração (2 pontos)**
  - O vídeo deve permitir identificar claramente o que está sendo testado e demonstrado.
  - Não é necessário edição ou narrativa elaborada, mas a demonstração deve ser compreensível.
  - Vídeos confusos, acelerados ou que não permitam validar o funcionamento das funcionalidades perderão pontuação.

Observação

Este item não avalia qualidade de edição, design visual ou complexidade técnica da solução. Avalia exclusivamente a **existência de um aplicativo funcional**, capaz de executar os fluxos propostos de forma real. Funcionalidades não demonstradas, fluxos quebrados ou soluções que apenas "atendem aos requisitos no papel" serão consideradas não concluídas para fins de avaliação.

Penalidades

As penalidades abaixo são cumulativas e podem resultar em nota final zero na Sprint, independentemente da pontuação obtida nos critérios avaliativos.

I. **Não realizar a entrega via GitHub Classroom (-20 pontos)**
A entrega deve ser feita exclusivamente pelo repositório oficial do GitHub Classroom.
Entregas fora deste padrão não serão consideradas válidas.

II. **Ausência de vídeo de apresentação (-50 pontos)**
A não entrega do vídeo, ou a entrega em formato diferente do especificado (ex.: link privado, vídeo inacessível ou hospedado fora do YouTube), implica penalização integral deste item.

III. **Aplicativo não funcional (-100 pontos)**
Será atribuída nota zero na Sprint caso o aplicativo:
- Não inicie corretamente (erro de build, crash ao abrir)
- Não permita acesso às funcionalidades principais
- Apresente falhas que impeçam a demonstração dos requisitos avaliativos
- Dependa de ajustes manuais, configurações externas não documentadas ou intervenção do professor para funcionar
- Não represente uma versão utilizável do produto proposto

IV. **Vídeo de apresentação incompatível com o aplicativo entregue (-100 pontos)**
Será atribuída nota zero na Sprint caso o vídeo de apresentação:
- Não demonstre o aplicativo real entregue no repositório
- Utilize protótipos, maquetes ou ferramentas de design (Figma, Adobe XD, etc.)
- Demonstre uma versão diferente da publicada e avaliada
- Não permita verificar, de forma inequívoca, a correspondência entre vídeo, código e versão publicada

MOBILE APP DEVELOPMENT 10/11

Penalidades

As penalidades abaixo são cumulativas e podem resultar em nota final zero na Sprint, independentemente da pontuação obtida nos critérios avaliativos.

V. Aplicativo publicado inacessível ou inconsistente (-40 pontos)

Aplica-se quando:

- O aplicativo não estiver disponível no Firebase App Distribution
- O professor não estiver adicionado como tester
- A versão publicada não corresponder ao código-fonte entregue
- Não for possível acessar ou instalar o aplicativo para avaliação

VI. Ausência de implementação real de Push Notification ou Local Notification (-40 pontos)

Aplica-se quando:

- A funcionalidade não estiver implementada
- A notificação existir apenas no código, sem possibilidade real de disparo
- O cenário apresentado não tiver relação com o contexto do aplicativo
- A funcionalidade não for demonstrada no vídeo

VII. Uso inadequado ou irrelevante do Oracle APEX (-50 pontos)

Aplica-se quando o Oracle APEX não for parte essencial da solução final.

Inclui, mas não se limita a:

- Uso do APEX apenas como banco de dados
- APIs sem processamento de dados ou lógica associada
- APIs não utilizadas pelo aplicativo
- Solução que poderia existir integralmente sem o APEX

Esta penalidade pode ser aplicada independentemente da existência formal de APIs.

MOBILE APP DEVELOPMENT 11/11

Penalidades

As penalidades abaixo são cumulativas e podem resultar em nota final zero na Sprint, independentemente da pontuação obtida nos critérios avaliativos.

VIII. Integrações simuladas ou incompletas (-40 pontos)

Aplica-se quando for identificado qualquer um dos seguintes casos:

- Uso de API apenas para operações de leitura (GET)
- CRUD incompleto ou parcialmente simulado
- Código de integração não utilizado pela interface
- Dados retornados pela API ignorados ou substituídos por valores fixos ou mockados
- Necessidade de reiniciar o aplicativo para refletir alterações de dados
- Atualização manual da interface após requisições sem controle de estado adequado

Esta penalidade cobre todas as tentativas de “fingir integração”.

IX. Navegação simulada ou inexistente (-30 pontos)

Aplica-se quando a navegação não for baseada em rotas reais. Inclui:

- Troca de componentes dentro da mesma tela como substituto de navegação
- Exibição ou ocultação de conteúdo sem troca de rota
- Uso de modais como substituto de telas
- Fluxos baseados em dados estáticos ou mockados

X. Arquitetura inadequada ou código não manutenível (-30 pontos)

Aplica-se quando o projeto apresentar problemas estruturais graves, como:

- Toda a lógica concentrada em componentes de tela
- Arquivos com múltiplas responsabilidades
- Repetição excessiva de código
- Estrutura que dificulte leitura, manutenção ou evolução

MOBILE APP DEVELOPMENT 12/12

Penalidades

As penalidades abaixo são cumulativas e podem resultar em nota final zero na Sprint, independentemente da pontuação obtida nos critérios avaliativos.

XI. Remoção ou descaracterização de funcionalidades entregues em sprints anteriores (-100 pontos)

Não é permitido remover, desativar ou inutilizar telas, fluxos ou funcionalidades já entregues e avaliadas nas sprints anteriores.
Esta penalidade implica **nota zero na Sprint**.

XII. Histórico de commits incoerente ou artificial (-50 pontos)

É esperado que o repositório represente a evolução real do projeto.
Inclui penalização quando houver:

- Poucos commits grandes
- Commits genéricos
- Upload único do projeto
- Histórico que não condiz com um desenvolvimento gradual ao longo das sprints

Observações Gerais

- As penalidades são cumulativas
- A aplicação de uma penalidade não exige a ocorrência de todos os itens listados
- Um único caso representativo é suficiente para aplicação